

CHALLENGE BRIEF

Internal 3-Day Learning Sprint — Revised Project Scope (No LLM / No Paid AI Services)

Project: Merchant Commerce Platform — User Profiles & Store Analytics Dashboard

3-day focus: extend the existing storefront/admin foundation with (1) richer customer user profiles and (2) an admin analytics dashboard showing visits, orders and top products — built entirely with data already modeled in the platform, with no AI agent, no LLM API, and no external paid service required.

1. Why This Replaces the Original AI-Agent Brief

The original brief (kept in version history) targeted a customer-facing AI chat agent with baseline-vs-advanced LLM evaluation. That scope assumes ongoing access to a hosted or self-hosted LLM, evaluation tooling, and cost/latency measurement across two AI systems — none of which the author has hands-on experience with yet. Rather than start a 3-day build on unfamiliar infrastructure, this revised brief targets a scope built entirely on skills already held: relational data modeling, REST APIs, and dashboard/UI work. The AI agent becomes a deliberate **Phase 2** addition once this foundation is working and the author has had time to learn local/self-hosted model options without added cost.

2. The Real-World Problem

Who has the problem?

Merchants running the store who currently have no visibility into who their customers are, how customers behave before purchasing, or which products actually perform.

What is the bottleneck?

The platform already persists users, orders and products, but there is no merchant-facing view that turns that raw data into an understanding of the business: repeat customers, order history per customer, traffic to the storefront, and which products convert. Store owners are flying blind between the admin's raw tables and any real insight.

Why is solving it valuable?

A merchant who can see "who my customers are" and "what is actually selling" can make faster, better decisions (restocking, promotions, customer follow-up) without needing a separate analytics vendor. This is also the necessary data foundation for the future AI shopping agent, since customer profile data and product performance data are exactly what a recommendation agent will need later.

3. Three-Day Scope

Part A — Customer User Profiles

- A customer-facing profile page (storefront) showing account details, order history, and order status.
- An admin-facing customer detail view: profile info, lifetime order count, lifetime spend, last order date.
- Reuse and extend the existing `user`, `order`, and `order item` Prisma models and existing `users / orders` API modules — no new persistence engine, just new read endpoints and aggregation queries.

- Basic role-aware access: a customer sees only their own profile; an admin/staff role sees any customer.

Part B — Store Analytics Dashboard (Admin)

- Storefront page-view / visit tracking: a lightweight event (page viewed, product viewed) recorded server-side, with no third-party analytics SaaS involved.
- Admin dashboard with: visits over time, orders over time, revenue over time, top products by orders/revenue, and new-vs-returning customers.
- Charts rendered in the existing Angular admin app (a plain charting library, e.g. Chart.js/ngx-charts — pick whichever is fastest to wire up, no new backend service).
- Data sourced only from the existing database (orders, orderitem, product, plus one new lightweight visit/event table).

4. What's Explicitly Out of Scope for These 3 Days

- No AI/LLM chat agent, no baseline-vs-advanced AI comparison, no prompt engineering, no agent trajectories.
- No third-party analytics platform (e.g. GA, Segment) — visits are tracked in-house, in the existing database.
- No SMS/loyalty, no social-media automation, no product-intelligence research agent.
- No new infrastructure/services beyond the existing NestJS API + Angular admin/storefront + Prisma/MySQL stack.

5. Proposed Data Model Additions

Minimal additions on top of the existing schema (see `prisma/schema.prisma`):

Model	Purpose	Key fields
storevisit (new)	Record a storefront page view / product view event	sessionId, userId? (nullable for anonymous), path, productId
user (existing)	Customer identity, reused as-is	id, email, firstName, lastName, createdAt, lastLoginAt
order / orderitem (existing)	Source of order history, revenue, top products	userId, status, grandTotal, placedAt, productId, quantity

This keeps the change surface small: one new table plus new aggregation/read endpoints. No changes to checkout, payments, or catalog write paths are required.

6. Three-Day Plan

Day	Focus	Deliverable
Day 1	Data + API	Add storevisit model + migration; add visit-tracking endpoint/middleware; add customer-profile endpoint
Day 2	Admin dashboard UI	Build the admin analytics dashboard screen with charts (visits, orders, revenue, top products)
Day 3	Storefront profile + polish + e2e	Build the customer-facing profile/order-history page; wire visit tracking into real storefront navigation

7. Definition of Done

- A merchant/admin can open a dashboard and see real visit, order, and revenue trends plus a top-products list, computed from real data in the database (not mock/static numbers).
- A merchant/admin can open any customer and see their profile plus real order history and lifetime value.
- A logged-in customer can view their own profile and past orders on the storefront.
- Visit tracking is demonstrably recording real events when the storefront is browsed (visible in the dashboard shortly after browsing).
- Everything runs against the existing Prisma/MySQL database with no new external services, credentials, or paid APIs.

8. Reproducibility Requirements

- README with exact commands to run migrations, seed data, and start API/admin/storefront (e.g. via Nx targets such as `npx nx run api:serve`, `npx nx run admin:serve`, `npx nx run storefront:serve`).
- Seed script or documented steps to generate enough sample orders/visits so charts are non-empty out of the box.
- Screenshots or a short screen recording of the dashboard and profile pages as evidence of the working feature.

9. Phase 2 (Deferred, Not Part of This 3-Day Scope)

Once this foundation is stable and the author has learned enough about local/self-hosted model options to avoid unnecessary API cost, the original AI shopping-assistant concept becomes a follow-on phase: a customer-facing chat agent that uses the now-existing customer profile and order/analytics data (in addition to the catalog) to make grounded, explainable product recommendations, evaluated with a baseline-vs-advanced comparison as in the original brief.

- Phase 2a: Baseline LLM + catalog-prompt shopping assistant (simple prompt, no retrieval).
- Phase 2b: Advanced agentic workflow — needs extraction, catalog retrieval, ranking, grounding/verification.
- Phase 2c: Evaluation harness with 10+ fixed test cases, scored for relevance, groundedness, and explanation quality.
- Model choice deferred until local/self-hosted options are evaluated for cost; a low-cost hosted API may be used as an interim fallback if local hosting isn't ready in time.

10. One-Sentence Challenge (Revised)

In three days, give merchants and customers real visibility into who is buying, what they're buying, and how the store is performing — built entirely on the existing data foundation, with no AI dependency — as the platform on which a future AI shopping agent will stand.

Note: This brief supersedes the original micro1 Agentic Workflows Hackathon AI-agent scope for the author's current 3-day learning sprint. It is a self-directed scope change, not an official hackathon submission document.